

Documentación MuMax3 AFNC

Víctor Costilla López

Versión 24.11

Contenidos

1. Introducción	1
1.1. Comentarios previos	1
1.2. Vista general del proyecto	2
2. TorqueFnAFNC	4
2.1. AddExchangeFieldAFNC	5
2.1.1. Variables para el cálculo del intercambio	6
2.1.2. AddExchangeAFNCCell	6
2.1.3. AddExchangeAFNCll	6
2.1.4. AddDMIAFNC	7
2.1.5. Compactación de las expresiones de la DMI	8
2.1.6. Implementación de la DMI	9
2.2. AddAnisotropyField	9
3. Resto de funciones en core	9
4. Archivo run y solvers	9
Referencias	10

1. Introducción

En este documento se pueden encontrar las modificaciones hechas al código de ALESTE2, basado en MUMAX3, con objetivo de introducir un tercer vector magnetización para estudiar los materiales antiferromagnéticos no colineales (AFNC).

1.1. Comentarios previos

El programa calcula la evolución de la magnetización reducida $\vec{m}(\vec{r}, t)$ en cada punto (celda) del espacio. Para ello resuelve la ecuación diferencial

$$\frac{\partial \vec{m}}{\partial t} = \vec{\tau} \quad (1.1)$$

Lo importante desde el punto de vista del funcionamiento del programa, es que consta de dos partes. Un *solver* para obtener la solución numérica de dicha ecuación y una función *torque* para calcular el valor de $\vec{\tau}$ en cada iteración del solver.

En realidad así funciona MUMAX3 original, en nuestro caso se resuelven tres ecuaciones

$$\frac{\partial \vec{m}_i}{\partial t} = \vec{\tau}_i \quad (1.2)$$

donde ahora cada $\vec{\tau}_i$ depende de las tres magnetizaciones. Se dedicará la [Sección 2](#) al torque, localizado en el archivo `vicost/src/github.com/MuMax3/3/engine/ext_AF+NC_core.go`. La [Sección 3](#) tratará de otras funciones que se han modificado en el archivo `core` y en la última sección se hablará de los nuevos solvers para el sistema de ecuaciones diferenciales propuesto.

Para moverse dentro del proyecto es útil entender qué se ha programado usando CUDA o Go. Los archivos de Go son los que trabajan a “nivel más alto” mientras que en los CUDA se realizan las operaciones. A lo que me refiero es a que toda la estructura del programa está escrita en Go, es decir, todas las llamadas a funciones y sus relaciones entre ellas. Si se siguen sus dependencias, siempre acaban en `k_funcion_async`, que es la forma de llamar al archivo CUDA donde realmente está definida la función, cuyo nombre será `funcion.cu` y se encontrará en la carpeta `cuda`.

Un último aspecto a resaltar es que se han mantenido todas las funcionalidades de *aleste2*. Esto se puede ver en los archivos `core` y `exchange`, ya que ambos usan la etiqueta `AF+NC`. Dichos archivos contienen todas las funciones de *aleste2* además de las nuevas. Se ha procedido de esta manera porque en *aleste2* ya hay dos vectores magnetización y se comparten muchas otras variables. Así no hace falta definir nuevos parámetros y complicar aún más la nomenclatura. Si se quiere simular antiferro, sólo hay que seleccionar un solver adecuado e igualar la tercera magnetización a cero.

1.2. Vista general del proyecto



Figura 1: Árbol de dependencias de la función torque en el archivo core. Tras cada función, su ruta desde `vicost/src/github.com/mumax/3`. Si no se indica ruta, está definida en core. En verde, archivos que no hizo falta modificar. En azul, archivos modificados. Y en rojo, archivos que faltan por implementar.

```

ext_AF+NC_core.go
├─ MagnetizationAFNC
│  └─ MagnetizationAFNC: cuda/ext\_AFNC\_AntiferroNC.go
│     └─ k_MagnetizationAFNC_async: cuda/ext\_AFNC\_MagnetizationAFNC.cu
├─ NeelnAFNC
│  └─ NeelnAFNC: cuda/ext\_AFNC\_AntiferroNC.go
│     └─ k_NeelnAFNC_async: cuda/ext\_AFNC\_Neel\_n\_AFNC.cu
├─ NeellAFNC
│  └─ NeellAFNC: cuda/ext\_AFNC\_AntiferroNC.go
│     └─ k_NeellAFNC_async: cuda/ext\_AFNC\_Neel\_l\_AFNC.cu
├─ GMagnetizationAFNC
│  └─ GMagnetizationAFNC: cuda/ext\_AFNC\_AntiferroNC.go
│     └─ k_GMagnetizationAFNC_async: cuda/ext\_AFNC\_GMagnetizationAFNC.cu
├─ GNeelnAFNC
│  └─ GNeelnAFNC: cuda/ext\_AFNC\_AntiferroNC.go
│     └─ k_GNeelnAFNC_async: cuda/ext\_AFNC\_GNeel\_n\_AFNC.cu
├─ GNeellAFNC
│  └─ GNeellAFNC: cuda/ext\_AFNC\_AntiferroNC.go
│     └─ k_GNeellAFNC_async: cuda/ext\_AFNC\_GNeel\_l\_AFNC.cu
├─ SetMFull1
├─ SetMFull2
└─ SetMFull3

```

Figura 2: Resto de funciones modificadas dentro del archivo core.

2. TorqueFnAFNC

La función torque devuelve tres campos vectoriales **dsti** (con $i=1,2,3$) que son los valores de $\vec{\tau}_i$ para cada celda del material y para un tiempo concreto. Se calculan considerando tres torques: el de Landau-Lifshitz ($\vec{\tau}_{LL}$) y los dos de transferencia de spin, Zhang-Li ($\vec{\tau}_{ZL}$) e Slonczewski ($\vec{\tau}_{SL}$) [1]. De estos tres, el único que depende explícitamente del campo magnético es el primero, cuya expresión es

$$\vec{\tau}_{LL,i} = \frac{\gamma_{LL,i}}{1 + \alpha_i^2} \left(\vec{m}_i \times \vec{B}_{\text{eff},i} + \alpha_i (\vec{m}_i \times \vec{B}_{\text{eff},i}) \right) \quad (2.1)$$

donde $\gamma_{LL,i}$ son los factores giromagnéticos para cada subred, α_i las constantes de amortiguación y $\vec{B}_{\text{eff},i}$ los campos magnéticos efectivos. Estos últimos son suma de los campos:

1. Externo, $\vec{B}_{\text{ext}}$
2. Desmagnetización (magnetostático), $\vec{B}_{\text{demag}}$
3. Intercambio de Heisenberg, $\vec{B}_{\text{exch}}$ [Secciones 2.1.2 y 2.1.3]
4. Intercambio de Dzyaloshinskii-Moriya (DMI), $\vec{B}_{\text{dmi}}$ [Sección 2.1.4]
5. Anisotropía magneto-cristalina, $\vec{B}_{\text{anis}}$ [Sección 2.2]
6. Térmico, $\vec{B}_{\text{therm}}$

Sólo se han modificado ambos campos de intercambio y el de anisotropía. Los primeros, por nuevas interacciones que surgen entre las subredes. Y el último, por considerar anisotropías tetragonal y hexagonal a mayores de las uniaxial y cúbica ya implementadas.

Aunque las expresiones del resto de campos no se han modificado, si se han hecho cambios en *core* cuando se llaman. Se enumeran a continuación.

Comentario (Sobre el uso de los **dsti**). Al comienzo de esta sección se ha atribuido a cada **dsti** un $\vec{\tau}_i$, pero en el cálculo de campos se les atribuye $\vec{B}_{\text{eff},i}$. Esto sucede porque estos objetos se reutilizan para el cómputo de ambas magnitudes dentro de la función torque. Primero se usan para obtener el campo efectivo en cada celda, después se calcula $\vec{\tau}_{LL,i}(\vec{m}_i, \vec{B}_{\text{eff},i})$ con ellos como argumento. Los demás torques no requieren los campos efectivos así que el valor de $\vec{\tau}_{LL,i}$ se almacena sobrescribiendo $\vec{B}_{\text{eff},i}$ en **dsti**.

Campo externo

Se ejecuta `B_ext.AddTo(dsti)` para cada subred. Simplemente añade el valor del campo externo aplicado para cada celda del espacio en un tiempo dado.

Campo de desmagnetización

Se ejecuta

```
SetDemagField(dst1)
data.Copy(dst2,dst1)
data.Copy(dst3,dst1)
```

Con esto se consigue tener el mismo campo en todas las subredes. Se calcula sólo una vez y se copia para cada una de ellas. Podemos hacerlo de esta manera porque es el primer sumando del campo efectivo, y al copiar no perdemos la información de las otras subredes. Esta misma idea se podría aplicar para el externo, pero sumar campos vectoriales es más sencillo que calcular este.

Campo térmico

Este campo se introduce mediante `B_therm.AddToAFNC(dst1, dst2, dst3)`. Donde la función `AddToAFNC(dst1, dst2, dst3)` llama a `updateAFNC(i)` con $i=1,2,3$. Esta última función es idéntica a la versión de *aleste2*, pero incluye un tercer valor de i . Dicha variable solo determina que $M_{\text{sat},i}$ y α_i se utilizan en el cómputo del campo.

2.1. AddExchangeFieldAFNC

La expresión para la densidad de energía de intercambio [2] (siguiendo el convenio de Einstein) es

$$\varepsilon_{\text{exch}} = A_{ii} \|\vec{\nabla} \vec{m}_i\|^2 + A_{ij} \langle \vec{\nabla} \vec{m}_i, \vec{\nabla} \vec{m}_j \rangle + B_{ij} \vec{m}_i \vec{m}_j \quad (i \neq j) \quad (2.2)$$

donde A_{ij} y B_{ij} son constantes que regulan las interacciones entre magnetizaciones de distintas celdas y de la misma celda respectivamente. De esta definición se obtienen los siguientes campos de intercambio

$$\vec{B}_{\text{exch},i} = \underbrace{\frac{B_{ij}}{M_{s,i}} \vec{m}_j}_{\text{ExchCell}} - \underbrace{\frac{2A_{ii}}{M_{s,i}} \nabla^2 \vec{m}_i}_{\text{Exch}} - \underbrace{\frac{A_{ij}}{M_{s,i}} \nabla^2 \vec{m}_j}_{\text{Exchl}} \quad (i \neq j) \quad (2.3)$$

donde $M_{s,i}$ son las magnetizaciones de saturación para cada subred. Esta ecuación se ha implementado separando cada sumando en una función distinta. El segundo de ellos corresponde a la función `AddExchange` de MUMAX3 original, el resto son nuevas.

También se incluye en esta función la contribución de DMI, su densidad de energía [3] es

$$\varepsilon_{\text{dmi},i} = D_i \left[(\vec{m}_i \cdot \vec{u}_n) \vec{\nabla} \vec{m}_i - \vec{m}_i \cdot \vec{\nabla} (\vec{m}_i \cdot \vec{u}_n) \right] \quad (2.4)$$

donde D_i son constantes definidas para cada subred. Y $\vec{u}_n$ es un vector unitario perpendicular a la interfaz entre las láminas de acople espín-órbita fuerte y ferromagnética. En el código se ha escogido la dirección z . El campo correspondiente es

$$\vec{B}_{\text{dmi},i} = \frac{2D_i}{M_{s,i}} \left[\vec{\nabla} (\vec{m}_i \cdot \vec{u}_n) - (\vec{\nabla} \cdot \vec{m}_i) \vec{u}_n \right] \quad (2.5)$$

Existe otro tipo de DMI que no es interfacial, en el código se llama `DMIBulk` y no se ha implementado. Tiene asociadas otras constantes $D_{\text{bulk},i}$. La función `AddExchangeFieldAFNC` añade todos estos campos a sus correspondientes `dsti` de la siguiente forma¹

```
switch
case !inter1 && !bulk1:
    cuda.AddExchange(dst1)
    cuda.AddExchange(dst2)
    cuda.AddExchange(dst3)
    cuda.AddExchangeAFNCCell(dst1, dst2, dst3)
    cuda.AddExchangeAFNC1l(dst1, dst2, dst3)
case inter1 && !bulk1:
    cuda.AddDMIAFNC(dst1, dst2, dst3)
    cuda.AddExchangeAFNCCell(dst1, dst2, dst3)
case bulk1 && !inter1:
    cuda.AddDMIBulk(dst1)
    cuda.AddDMIBulk(dst2)
    cuda.AddDMIBulk(dst3)
    cuda.AddExchangeAFNCCell(dst1, dst2, dst3)
    cuda.AddExchangeAFNC1l(dst1, dst2, dst3)
case inter1 && bulk1:
    util.Fatal
```

donde las variables `inter1` y `bulk1` comprueban si las constantes D_1 y $D_{\text{bulk},1}$ son distintas de cero, respectivamente. Lo hacen para la primera subred, así que sus constantes deciden que interacciones se calcularán. No se pueden dar ambas DMI a la vez. Solo nos interesan los primeros casos: sin y con DMI interfacial, sin DMI en volumen.

¹Este texto es una especie de pseudocódigo, la sintaxis no es correcta y faltan argumentos. Esto será cierto para la mayoría de “códigos” en la documentación.

2.1.1. Variables para el cálculo del intercambio

La correspondencia entre las constantes de la Ecuación 2.3 y las variables usadas es

$$B_{ij} \rightarrow \text{Bexij} \quad A_{ii} \rightarrow \text{Aexi} \rightarrow \text{lex2i} \quad A_{ij} \rightarrow \text{Aexllij} \rightarrow \text{lexllij} \quad (2.6)$$

donde ambos `lex` son `exchParam` dentro del código, y estos parámetros son los que se introducen como argumento a las funciones. El carácter 2 no es una errata, se ha heredado del código original de MUMAX3.

Sucede que $A_{ij} = A_{ji}$, por tanto sólo hay definidos tres `lexllij`. En el caso de `Bexij` sí están definidas todas las combinaciones, aunque creemos que no es necesario, pero también se ha heredado y por conveniencia no se cambió (solo hace falta igualar sus valores al ejecutar el programa). De manera similar, para DMI se usan

$$D_i \rightarrow \text{Dindi} \rightarrow \text{din2i} \quad D_{\text{bulk},i} \rightarrow \text{Dbulki} \rightarrow \text{dbulk2i} \quad (2.7)$$

2.1.2. AddExchangeAFNCCell

El primer sumando de la Ecuación 2.3 calcula el campo debido a la magnetización de las otras subredes evaluadas en la misma celda en que se está calculando. Se ha definido la función `AddExchangeAFNCCell`, que añade a cada `dsti`

```
dst1x[i] += invMs1*bex12*m2x[i] + invMs1*bex13*m3x[i];
dst1y[i] += invMs1*bex12*m2y[i] + invMs1*bex13*m3y[i];
dst1z[i] += invMs1*bex12*m2z[i] + invMs1*bex13*m3z[i];
```

Solo se han escrito los términos para la primera subred, es fácil ver la forma para las demás atendiendo a la Ecuación 2.3. Se ha abandonado la notación de i -ésima subred porque en este código dicho índice se reserva para designar la celda en que se está trabajando²

2.1.3. AddExchangeAFNCII

El tercer sumando de la Ecuación 2.3 relaciona las magnetizaciones de distintas subredes y de diferentes celdas, en particular, de los vecinos ortogonales. La forma funcional de este sumando es casi idéntica al segundo, que ya está implementado en MUMAX3. Por lo tanto, con solo cambiar los argumentos de la función original se obtiene el resultado esperado

```
//Sublattice 1
k_addexchange_async(dst1, m2, ms1, llex12)
k_addexchange_async(dst1, m3, ms1, llex13)
//Sublattice 2
k_addexchange_async(dst2, m1, ms2, llex12)
k_addexchange_async(dst2, m3, ms2, llex23)
//Sublattice 3
k_addexchange_async(dst3, m1, ms3, llex13)
k_addexchange_async(dst3, m2, ms3, llex23)
```

En cada llamada de la función `addexchange` se computa la divergencia con las magnetizaciones de otra subred, utilizando la constante A_{ij} adecuada y con $M_{s,i}$ de la red para la cual se calcula el campo.

²No realmente, este índice tiene que ver con “hilos” y “bloques” de la GPU. No se va a entrar en esas consideraciones.

2.1.4. AddDMIAFNC

Una versión más explicita de la Ecuación 2.5 es

$$\vec{B}_{\text{dmi},i} = \frac{2D_i}{M_{s,i}} (\partial_x m_{i,z}, \partial_y m_{i,z}, -\partial_x m_{i,x} - \partial_y m_{i,y}) \quad (2.8)$$

Para calcular derivadas se necesita conocer la magnetización de los vecinos, pero si alguno de ellos se encuentra fuera de los límites de la muestra, habrá que extrapolarlo. Para ello se imponen las siguientes condiciones de contorno

$$\vec{m}_i \times \left[\left(2A_{ii}(\vec{\nabla}\vec{m}_i) + A_{ij}(\vec{\nabla}\vec{m}_j) \right) \cdot \hat{n} + D_i \vec{m}_i \times (\hat{n} \times \vec{u}_z) \right] = 0 \quad (i \neq j) \quad (2.9)$$

donde $\hat{n}$ es un vector unitario perpendicular a la superficie del borde que se está considerando. Se van a imponer unas condiciones menos generales, en particular, que todo el corchete de la Ecuación 2.9 sea nulo. Con esas condiciones se obtienen los siguientes resultados:

Bordes X

$$\begin{aligned} \partial_x m_{1,x} = & - \frac{1}{2A_{11} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right) - \frac{(A_{13} - \frac{A_{12}A_{23}}{2A_{22}})^2}{2A_{33} \left(1 - \frac{(A_{23})^2}{4A_{22}A_{33}} \right)}} \left[D_1 m_{1,z} + \left(\frac{A_{23} \left(A_{13} - \frac{A_{12}A_{23}}{2A_{22}} \right)}{4A_{22}A_{33} \left(1 - \frac{(A_{23})^2}{4A_{22}A_{33}} \right)} - \frac{A_{12}}{2A_{22}} \right) D_2 m_{2,z} \right. \\ & \left. - \left(\frac{A_{13} - \frac{A_{12}A_{23}}{2A_{22}}}{2A_{33} \left(1 - \frac{(A_{23})^2}{4A_{22}A_{33}} \right)} \right) D_3 m_{3,z} \right] \end{aligned} \quad (2.10)$$

$$\begin{aligned} \partial_x m_{2,x} = & - \frac{1}{2A_{22} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right) - \frac{(A_{23} - \frac{A_{12}A_{13}}{2A_{11}})^2}{2A_{33} \left(1 - \frac{(A_{13})^2}{4A_{11}A_{33}} \right)}} \left[\left(\frac{A_{13} \left(A_{23} - \frac{A_{12}A_{13}}{2A_{11}} \right)}{4A_{11}A_{33} \left(1 - \frac{(A_{13})^2}{4A_{11}A_{33}} \right)} - \frac{A_{12}}{2A_{11}} \right) D_1 m_{1,z} + D_2 m_{2,z} \right. \\ & \left. - \left(\frac{A_{23} - \frac{A_{12}A_{13}}{2A_{11}}}{2A_{33} \left(1 - \frac{(A_{13})^2}{4A_{11}A_{33}} \right)} \right) D_3 m_{3,z} \right] \end{aligned} \quad (2.11)$$

$$\begin{aligned} \partial_x m_{3,x} = & - \frac{1}{2A_{33} \left(1 - \frac{(A_{13})^2}{4A_{11}A_{33}} \right) - \frac{(A_{23} - \frac{A_{12}A_{13}}{2A_{11}})^2}{2A_{22} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right)}} \left[\left(\frac{A_{12} \left(A_{23} - \frac{A_{12}A_{13}}{2A_{11}} \right)}{4A_{11}A_{22} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right)} - \frac{A_{13}}{2A_{11}} \right) D_1 m_{1,z} \right. \\ & \left. - \left(\frac{A_{23} - \frac{A_{12}A_{13}}{2A_{11}}}{2A_{11} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right)} \right) D_2 m_{2,z} + D_3 m_{3,z} \right] \end{aligned} \quad (2.12)$$

$$\partial_x m_{1,y} = \partial_x m_{2,y} = \partial_x m_{3,y} = 0 \quad (2.14)$$

$$\partial_x m_{1,z} = -\partial_x m_{1,x} \quad [x \leftrightarrow z] \quad (2.15)$$

$$\partial_x m_{2,z} = -\partial_x m_{2,x} \quad [x \leftrightarrow z] \quad (2.16)$$

$$\partial_x m_{3,z} = -\partial_x m_{3,x} \quad [x \leftrightarrow z] \quad (2.17)$$

Los corchetes en las tres últimas ecuaciones significan sustituir las componentes x de m_i por sus componentes z en el lado derecho de la igualdad.

Bordes Y

$$\partial_y m_{1,x} = \partial_y m_{2,y} = \partial_y m_{3,y} = 0 \quad (2.18)$$

$$\partial_y m_{1,y} = \partial_x m_{1,x} \quad (2.19)$$

$$\partial_y m_{2,y} = \partial_x m_{2,x} \quad (2.20)$$

$$\partial_y m_{3,y} = \partial_x m_{3,x} \quad (2.21)$$

$$\partial_y m_{1,z} = -\partial_x m_{1,x} \quad [x \leftrightarrow z] \quad (2.22)$$

$$\partial_y m_{2,z} = -\partial_x m_{2,x} \quad [x \leftrightarrow z] \quad (2.23)$$

$$\partial_y m_{3,z} = -\partial_x m_{3,x} \quad [x \leftrightarrow z] \quad (2.24)$$

Bordes Z

$$\partial_z m_{1,x} = \partial_z m_{2,x} = \partial_z m_{3,x} = 0 \quad (2.25)$$

$$\partial_z m_{1,y} = \partial_z m_{2,y} = \partial_z m_{3,y} = 0 \quad (2.26)$$

$$\partial_z m_{1,z} = \partial_z m_{2,z} = \partial_z m_{3,z} = 0 \quad (2.27)$$

Donde las ecuaciones de los bordes z además de 2.14 y 2.18 son ciertas si se cumple

$$\left(1 - \frac{\left(A_{13} - \frac{A_{12}A_{23}}{2A_{22}} \right)^2}{4A_{11}A_{33} \left(1 - \frac{(A_{12})^2}{4A_{11}A_{22}} \right) \left(1 - \frac{(A_{23})^2}{4A_{22}A_{33}} \right)} \right) \neq 0 \quad (2.28)$$

2.1.5. Compactación de las expresiones de la DMI

Aprovechando la repetición de ciertos términos en las ecuaciones anteriores, se han implementado definiendo funciones intermedias que calculen cada uno de ellos. Por conveniencia, se prescinde de un índice y nos referiremos a las constantes A siguiendo este criterio

$$\begin{array}{lll} A_{11} \rightarrow A_1 & A_{22} \rightarrow A_2 & A_{33} \rightarrow A_3 \\ A_{12} \rightarrow A_4 & A_{13} \rightarrow A_5 & A_{23} \rightarrow A_6 \end{array} \quad (2.29)$$

Las funciones intermedias son las siguientes

$$\text{mid1} = A_5 - \frac{A_4 A_6}{2A_2} \quad (2.30)$$

$$\text{mid2} = 2A_3 \left(1 - \frac{(A_6)^2}{4A_2 A_3} \right) \quad (2.31)$$

$$\text{pref} = \frac{1}{2A_1 \left(1 - \frac{(A_4)^2}{4A_1 A_2} \right) - \frac{\left(A_5 - \frac{A_4 A_6}{2A_2} \right)^2}{2A_3 \left(1 - \frac{(A_6)^2}{4A_2 A_3} \right)}} = \frac{1}{2A_1 \left(1 - \frac{(A_4)^2}{4A_1 A_2} \right) - \frac{(\text{mid1})^2}{\text{mid2}}} \quad (2.32)$$

$$\text{coef1} = \frac{A_6 \left(A_5 - \frac{A_4 A_6}{2A_2} \right)}{4A_2 A_3 \left(1 - \frac{(A_6)^2}{4A_2 A_3} \right)} - \frac{A_4}{2A_2} = \frac{A_6 \cdot \text{mid1}}{2A_2 \cdot \text{mid2}} - \frac{A_4}{2A_2} \quad (2.33)$$

$$\text{coef2} = -\frac{A_5 - \frac{A_4 A_6}{2A_2}}{2A_3 \left(1 - \frac{(A_6)^2}{4A_2 A_3} \right)} = -\frac{\text{mid1}}{\text{mid2}} \quad (2.34)$$

Todas estas funciones están definidas en `cuda/dmiafnc.h`

Estas funciones son simplemente otra forma de escribir la condición de contorno $\partial_x m_{1,x}$

$$\partial_x m_{1,x} = -\text{pref} \cdot (D_1 m_{1,z} + \text{coef1} \cdot D_2 m_{2,z} + \text{coef2} \cdot D_3 m_{3,z}) \quad (2.35)$$

Lo interesante es que las otras dos condiciones que necesitamos saber $\partial_x m_{2,x}$ y $\partial_x m_{3,x}$ se pueden escribir de manera muy similar si cambiamos el orden de las variables A . Si entendemos estas funciones como

```
pref(A1,A2,A3,A4,A5,A6)
coef1(A1,A2,A3,A4,A5,A6)
coef2(A1,A2,A3,A4,A5,A6)
```

Y para cada subred introducimos las constantes en los argumentos siguiendo esta tabla

	$\partial_x m_{1,x}$	$\partial_x m_{2,x}$	$\partial_x m_{3,x}$
pref	123456	213465	312564
coef1	123456	213465	312564
coef2	123456	213465	312564

Entonces las otras condiciones de contorno toman la forma

$$\partial_x m_{1,x} = -\text{pref} \cdot (\text{coef1} \cdot D_1 m_{1,z} + D_2 m_{2,z} + \text{coef2} \cdot D_3 m_{3,z}) \quad (2.36)$$

$$\partial_x m_{1,x} = -\text{pref} \cdot (\text{coef1} \cdot D_1 m_{1,z} + \text{coef2} \cdot D_2 m_{2,z} + D_3 m_{3,z}) \quad (2.37)$$

Como el resto de condiciones son las mismas cambiando $m_{i,z} \rightarrow m_{i,x}$ o nulas, con esto hemos compactado todas las expresiones. De manera resumida:

	D_1	D_2	D_3	Orden
m_1	1	C_1	C_2	123456
m_2	C_1	1	C_2	213465
m_3	C_1	C_2	1	312564

2.1.6. Implementación de la DMI

Como se puede ver en la estructura de la función `AddExchangeFieldAFNC` en la Sección 2.1, en el caso de DMI interfacial no se llama a `AddExchange`. Esto es porque dentro de la DMI también se calcula el intercambio usual, ya que estamos calculando el valor de las magnetizaciones vecinas con las condiciones de contorno (si son nulas). Y con estas magnetizaciones vecinas se computan las derivadas direccionales necesarias para ambas interacciones.

2.2. AddAnisotropyField

3. Resto de funciones en core

4. Archivo run y solvers

Referencias

- [1] Arne Vansteenkiste et al. “The design and verification of MuMax3”. En: *AIP Advances* 4.10 (oct. de 2014), pág. 107133. ISSN: 2158-3226. DOI: [10.1063/1.4899186](https://doi.org/10.1063/1.4899186). eprint: https://pubs.aip.org/aip/adv/article-pdf/doi/10.1063/1.4899186/12878560/107133\1_online.pdf. URL: <https://doi.org/10.1063/1.4899186>.
- [2] Luis Sánchez-Tejerina et al. “Dynamics of domain-wall motion driven by spin-orbit torque in antiferromagnets”. En: *Phys. Rev. B* 101 (1 2020), pág. 014433. DOI: [10.1103/PhysRevB.101.014433](https://doi.org/10.1103/PhysRevB.101.014433). URL: <https://link.aps.org/doi/10.1103/PhysRevB.101.014433>.
- [3] Luis Sánchez-Tejerina San José. “Modelización de nanodispositivos magnéticos con especial énfasis en los fenómenos de acoplamiento espin-órbita”. Tesis doct. 2018.